Kegiatan Belajar 4F.2: Baca-Tulis JSON dan Proteksi Folder Data

Hari 13 · 8 JP · UK 1.2, 1.3, 1.4, dan 1.5 (utama)

Tujuan KB 4F.2. Setelah mengikuti KB ini, Anda mampu membaca dan menulis berkas JSON dari PHP, memusatkan logika itu ke dalam fungsi yang dipakai berulang, melindungi folder `data/` dari akses langsung lewat browser, dan membangun endpoint yang membaca `produk.json`.

Kegiatan	JP
Baca-tulis JSON dari PHP: `file_get_contents`, `file_put_contents`, `json_decode`, `json_encode`	2
Praktik: `api/lib/json_store.php` — fungsi baca-tulis terpusat	2
Proteksi folder `data/` lewat `.htaccess`	2
Praktik: endpoint `api/produk.php`	2
Jumlah	**8**

Materi 1: Membaca dan Menulis JSON dari PHP

PHP menyediakan dua pasang fungsi bawaan yang menjadi inti seluruh Bagian B2:

Fungsi	Fungsi	Padanan di JavaScript (Modul 3)
`file_get_contents($path)`	Membaca seluruh isi berkas sebagai teks	`fetch()` + `.text()`
`json_decode($teks, true)`	Mengubah teks JSON menjadi array PHP	`JSON.parse()`
`json_encode($data)`	Mengubah array PHP menjadi teks JSON	`JSON.stringify()`
`file_put_contents($path, $teks)`	Menulis teks ke berkas (menimpa seluruh isi)	*(tidak ada padanan — browser tidak bisa menulis berkas)*

```php
<?php
// Membaca
$teks = file_get_contents("data/produk.json");
$data = json_decode($teks, true);   // true -> hasilnya array asosiatif, bukan objek

// Menulis (menimpa SELURUH isi berkas dengan data terbaru)
$teksBaru = json_encode($data, JSON_PRETTY_PRINT);
file_put_contents("data/produk.json", $teksBaru);
```

Perhatikan baik-baik: `file_put_contents` tidak "menambahkan satu baris" seperti `INSERT` pada basis data — ia **menimpa seluruh berkas** dengan apa pun yang Anda berikan. Karena itu, pola yang selalu dipakai sepanjang Bagian B2 adalah: **baca seluruh data → ubah di dalam PHP (tambah/ubah/hapus satu elemen array) → tulis ulang seluruhnya**. Ini berbeda dari pola `UPDATE ... WHERE id = ...` yang dipakai basis data, dan merupakan konsekuensi langsung dari memilih berkas datar sebagai penyimpanan.

`JSON_PRETTY_PRINT` membuat berkas JSON hasil tulisan tetap rapi dan mudah dibaca manusia (berguna saat Anda perlu membuka berkasnya langsung untuk memeriksa isinya) — tanpa opsi ini, hasilnya tetap sah tapi berupa satu baris panjang yang sulit dibaca.

Praktik 1: Fungsi Baca-Tulis Terpusat

Alih-alih menulis ulang `file_get_contents`/`json_decode` di setiap endpoint (dan berisiko ada yang lupa menanganinya dengan benar), buat **satu** berkas pusat.

Buat folder `api/lib/`, isi `api/lib/json_store.php`:

```
<?php
// json_store.php: fungsi terpusat untuk membaca dan menulis berkas JSON.
// Dipakai oleh SELURUH endpoint di Bagian B2 -- satu tempat, satu kebenaran.

function bacaJson(string $path): array {
    if (!file_exists($path)) {
        return [];   // berkas belum ada dianggap "data kosong", bukan galat
    }
    $teks = file_get_contents($path);
    $data = json_decode($teks, true);
    return $data ?? [];   // jika isi berkas rusak/bukan JSON valid, tetap kembalikan array kosong
}

function tulisJson(string $path, array $data): bool {
    $teks = json_encode($data, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES);
    // LOCK_EX: kunci berkas selama proses penulisan berlangsung, mencegah
    // dua proses menulis ke berkas yang sama secara bersamaan.
    return file_put_contents($path, $teks, LOCK_EX) !== false;
}
```

Catatan penting soal `tulisJson()` di atas. Flag **`LOCK_EX`** melindungi proses **penulisan itu sendiri** dari tertimpa proses lain yang menulis di saat bersamaan — ini sudah cukup untuk kebanyakan kebutuhan Modul 4F. Tapi untuk kasus yang lebih berisiko (dua orang checkout nyaris bersamaan, berebut stok yang sama) dibutuhkan pola yang **lebih ketat** — dibahas di KB 4F.4.

`json_decode($teks, true) ?? []` memakai operator `??` (*null coalescing*) — berarti "jika hasil di kiri adalah `null`, pakai nilai di kanan sebagai gantinya". Ini melindungi dari kasus berkas JSON yang isinya rusak (misalnya kehilangan tanda kurung) sehingga `json_decode` gagal dan menghasilkan `null` — alih-alih PHP ikut rusak karena memperlakukan `null` sebagai array, kita kembalikan array kosong.

Materi 2: Mengapa Folder `data/` Harus Diproteksi

Coba buka langsung di browser: `http://localhost/kopi-nusantara/data/produk.json`. Berkas itu **tampil apa adanya** — ini bukan masalah besar untuk `produk.json` (datanya memang publik), tapi akan menjadi masalah **serius** begitu Modul 6F menambahkan `pengguna.json` berisi kata sandi admin yang di-*hash*. Jika folder `data/` tidak diproteksi, siapa pun yang tahu alamatnya bisa mengunduh **seluruh isi** `pengguna.json` langsung lewat browser — risiko yang **tidak pernah ada** pada basis data sungguhan (basis data tidak pernah bisa diakses langsung lewat URL).

Solusi: satu berkas kecil bernama `.htaccess` di dalam folder `data/`, berisi perintah untuk Apache menolak **semua** permintaan langsung ke folder itu.

Praktik 2: Memproteksi Folder `data/`

Buat `data/.htaccess`:

```apache
# data/.htaccess
# Menolak SEMUA permintaan langsung ke folder ini lewat browser.
# PHP di luar folder ini (api/, admin/) TETAP BISA membaca berkas di sini
# secara normal -- larangan ini hanya berlaku untuk permintaan lewat URL.

Require all denied
```

`Require all denied` adalah sintaks Apache versi modern (2.4 ke atas, yang dipakai Laragon). Jika suatu saat Anda memakai server dengan Apache versi lama, baris yang setara adalah `Deny from all` — tapi untuk pelatihan ini `Require all denied` sudah tepat.

Uji sekarang:

1. Sebelum menyimpan `.htaccess`: buka `http://localhost/kopi-nusantara/data/produk.json` — berkasnya tampil.
2. Simpan `data/.htaccess` seperti di atas, segarkan alamat yang sama — browser sekarang menampilkan **"403 Forbidden"**.
3. Buka Kopi Nusantara (`index.html`) seperti biasa — halaman tetap tampil normal untuk sekarang (karena `app.js` masih memanggil `data/produk.json` langsung lewat `fetch()` — ini akan ikut diblokir setelah proteksi aktif, dan **sengaja** diperbaiki di KB 4F.3, bukan sekarang).

Poin ke-3 di atas penting dipahami: begitu `.htaccess` aktif, cara lama (`fetch("data/produk.json")` langsung dari `app.js`) **akan ikut berhenti berfungsi** karena browser juga tidak diizinkan mengaksesnya. Ini bukan kesalahan — ini justru **alasan** mengapa KB 4F.3 memindahkan `app.js` agar memanggil `api/produk.php`, yang membaca berkas itu dari **sisi server** (bukan lewat URL), sehingga tidak terkena proteksi `.htaccess` sama sekali.

Praktik 3: Endpoint `api/produk.php`

```php
<?php
require __DIR__ . "/lib/json_store.php";

header("Content-Type: application/json");

$data = bacaJson(__DIR__ . "/../data/produk.json");

echo json_encode($data);
```

Catatan desain — lebih sederhana dibanding versi basis data. Karena `data/produk.json` **sudah** berisi seluruh struktur (`site`, `menu`, **dan** `produk`) sejak Bagian A, endpoint ini cukup membaca dan meneruskan apa adanya — tidak perlu menyusun ulang `site`/`menu` secara manual di PHP seperti yang dilakukan versi basis data (di sana, hanya bagian `produk` yang berpindah ke tabel, sehingga `site`/`menu` harus ditulis ulang terpisah).

Uji endpoint ini: buka `http://localhost/kopi-nusantara/api/produk.php` di browser — isinya harus identik dengan apa yang sebelumnya tampil di `data/produk.json` sebelum folder itu diproteksi.

Latihan 2

1. Buka `data/produk.json` langsung lewat alamat browser **setelah** `.htaccess` aktif — pastikan muncul 403 Forbidden.
2. Buka `api/produk.php` di alamat yang sama — pastikan data **tetap tampil normal** (bukti bahwa PHP membaca berkas dari sisi server, bukan lewat URL).

3. Tambahkan satu produk baru **langsung dengan menyunting `data/produk.json`** di VS Code, simpan, segarkan `api/produk.php` — produk baru harus langsung muncul.
4. *Tantangan:* ubah `api/produk.php` agar menerima parameter `?kategori=arabika` lewat `$_GET`, dan jika ada, hanya mengembalikan produk dengan kategori itu (petunjuk: `array_filter()` pada array hasil `bacaJson()`).

Rangkuman KB 4F.2

- `file_get_contents`/`json_decode` membaca JSON; `file_put_contents`/`json_encode` menulis ulang **seluruh** berkas.
- Fungsi baca-tulis dipusatkan di `api/lib/json_store.php` agar tidak ditulis ulang (dan berpotensi salah) di banyak tempat.
- Folder `data/` diproteksi lewat `.htaccess` (`Require all denied`) — mencegah berkas sensitif diunduh langsung lewat URL.
- Begitu proteksi aktif, akses **langsung** dari browser ke berkas JSON berhenti bekerja — hanya PHP di sisi server yang masih bisa membacanya.

Cek Kemampuan KB 4F.2

- ☐ `data/.htaccess` aktif — mengakses `data/produk.json` langsung lewat browser menghasilkan **403 Forbidden**.
- ☐ `api/produk.php` tetap mengembalikan data dengan benar meski `data/` sudah diproteksi.
- ☐ Saya dapat menjelaskan mengapa proteksi ini tidak dibutuhkan pada versi basis data.
- ☐ Saya dapat menjelaskan fungsi `bacaJson()` dan `tulisJson()` tanpa melihat kode.

Kesalahan umum: `.htaccess` disimpan dengan nama salah (mis. `htaccess.txt` karena Windows menyembunyikan akhiran berkas); lupa `require` berkas `json_store.php` di endpoint; mengira galat 403 setelah memasang `.htaccess` adalah kesalahan — itu justru **bukti proteksi berhasil**.